1. Data Acquisition

In this section, we connect to different sources to retrieve the raw data.

In [2]:
```
# Import libraries for data handling
import pandas as pd
import numpy as np
import glob
import os
from datetime import datetime

# Import libraries for weather API
import requests
import openmeteo_requests
import requests_cache
from retry_requests import retry

# Import libraries for holidays
import holidays

# Import libraries for Selenium (web scraping)
!pip install selenium
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait, Select
from selenium.webdriver.support import expected_conditions as EC
import time
```

```
Requirement already satisfied: selenium in /opt/conda/lib/python3.11/site-package
s (4.39.0)
Requirement already satisfied: urllib3<3.0,>=2.5.0 in /opt/conda/lib/python3.11/s
ite-packages (from urllib3[socks]<3.0,>=2.5.0->selenium) (2.6.2)
Requirement already satisfied: trio<1.0,>=0.31.0 in /opt/conda/lib/python3.11/sit
e-packages (from selenium) (0.32.0)
Requirement already satisfied: trio-websocket<1.0,>=0.12.2 in /opt/conda/lib/pyth
on3.11/site-packages (from selenium) (0.12.2)
Requirement already satisfied: certifi>=2025.10.5 in /opt/conda/lib/python3.11/si
te-packages (from selenium) (2025.11.12)
Requirement already satisfied: typing_extensions<5.0,>=4.15.0 in /opt/conda/lib/p
ython3.11/site-packages (from selenium) (4.15.0)
Requirement already satisfied: websocket-client<2.0,>=1.8.0 in /opt/conda/lib/pyt
hon3.11/site-packages (from selenium) (1.9.0)
Requirement already satisfied: attrs>=23.2.0 in /opt/conda/lib/python3.11/site-pa
ckages (from trio<1.0,>=0.31.0->selenium) (25.4.0)
Requirement already satisfied: sortedcontainers in /opt/conda/lib/python3.11/site
-packages (from trio<1.0,>=0.31.0->selenium) (2.4.0)
Requirement already satisfied: idna in /opt/conda/lib/python3.11/site-packages (f
rom trio<1.0,>=0.31.0->selenium) (3.4)
Requirement already satisfied: outcome in /opt/conda/lib/python3.11/site-packages
(from trio<1.0,>=0.31.0->selenium) (1.3.0.post0)
Requirement already satisfied: sniffio>=1.3.0 in /opt/conda/lib/python3.11/site-p
ackages (from trio<1.0,>=0.31.0->selenium) (1.3.0)
Requirement already satisfied: wsproto>=0.14 in /opt/conda/lib/python3.11/site-pa
ckages (from trio-websocket<1.0,>=0.12.2->selenium) (1.3.2)
Requirement already satisfied: pysocks!=1.5.7,<2.0,>=1.5.6 in /opt/conda/lib/pyth
on3.11/site-packages (from urllib3[socks]<3.0,>=2.5.0->selenium) (1.7.1)
Requirement already satisfied: h11<1,>=0.16.0 in /opt/conda/lib/python3.11/site-p
ackages (from wsproto>=0.14->trio-websocket<1.0,>=0.12.2->selenium) (0.16.0)
```

1.1 Bike Data Acquisition

In [4]:
```
os.chdir("/home/jovyan/work/project_meloni_bigoni/notebook/post_22_12/citibike")
print(os.getcwd())

print("Acquiring Bike Data")
# Look for all CSV files with 'citibike' in the name
file_list = glob.glob("*-citibike-tripdata_*.csv")

# Ignore files that are results from previous runs
file_list = [f for f in file_list if "Analysis" not in f and "knicks" not in f]

print(f"Found {len(file_list)} bike data files.")
```

```
/home/jovyan/work/project_meloni_bigoni/notebook/post_22_12/citibike
Acquiring Bike Data
Found 53 bike data files.
```

1.1.1 Download raw data

codice per scaricare i file dal sito direttamante in locale

In [5]:
```
import requests
import xml.etree.ElementTree as ET

url = "https://s3.amazonaws.com/tripdata/"
xml = requests.get(url).content
```

```
root = ET.fromstring(xml)

start, end = 202411, 202510

zip_files = [
    key.text
    for key in root.findall(".//{http://s3.amazonaws.com/doc/2006-03-01/}Key")
    if key.text.endswith(".zip")
    and key.text[:6].isdigit()
    and start <= int(key.text[:6]) <= end
    and "-citibike-tripdata" in key.text
]

print(zip_files)
```

```
['202411-citibike-tripdata.zip', '202412-citibike-tripdata.zip', '202501-citibike
-tripdata.zip', '202502-citibike-tripdata.zip', '202503-citibike-tripdata.zip',
'202504-citibike-tripdata.zip', '202505-citibike-tripdata.zip', '202506-citibike-
tripdata.zip', '202507-citibike-tripdata.zip', '202508-citibike-tripdata.zip', '2
02509-citibike-tripdata.zip', '202510-citibike-tripdata.zip']
```

In [6]:

```
import os
import requests
from zipfile import ZipFile
from io import BytesIO

# --- CONFIGURAZIONE ---
download_folder = "citibike_data_python_automatic"
os.makedirs(download_folder, exist_ok=True)

# Lista dei file ZIP da scaricare
# Qui puoi mettere solo i file che ti servono, ad esempio 202401 → 202412
base_url = "https://s3.amazonaws.com/tripdata/"


# --- FUNZIONE PER SCARICARE E ESTRARE ZIP ---
def download_and_extract_zip(filename, folder):
    url = base_url + filename
    local_path = os.path.join(folder, filename)

    print(f"Scaricando {filename}...")
    r = requests.get(url, stream=True)
    r.raise_for_status()

    # Apri ZIP in memoria e salva CSV dentro folder
    with ZipFile(BytesIO(r.content)) as z:
        for file in z.namelist():
            if file.endswith(".csv"):
                print(f"  Estraendo {file}...")
                z.extract(file, folder)

# --- LOOP SU TUTTI I FILE ---
#for f in zip_files:
    #download_and_extract_zip(f, download_folder)

download_and_extract_zip(zip_files[0], download_folder)

print("Download e estrazione completati!")
```

```
Scaricando 202411-citibike-tripdata.zip...
  Estraendo 202411-citibike-tripdata_3.csv...
  Estraendo 202411-citibike-tripdata_2.csv...
  Estraendo 202411-citibike-tripdata_1.csv...
  Estraendo 202411-citibike-tripdata_4.csv...
Download e estrazione completati!
```

1.2 Weather Data Acquisition

We connect to Open-Meteo API to fetch the raw weather records.

In [ ]:
```
print("--- Acquiring Weather Data ---")
# Set up the API connection with caching
cache_session = requests_cache.CachedSession('.cache', expire_after = -1)
retry_session = retry(cache_session, retries = 5, backoff_factor = 0.2)
openmeteo = openmeteo_requests.Client(session = retry_session)

# API URL and parameters (we ask for a wide range to cover all potential bike da
url = "https://archive-api.open-meteo.com/v1/archive"
params = {
    "latitude": 40.7128, # New York
    "longitude": -74.0060,
    "start_date": "2024-11-01", # Broad range, we will filter later or it fits a
    "end_date": "2025-10-31",
    "hourly": ["temperature_2m", "relative_humidity_2m", "apparent_temperature",
    "timezone": "America/New_York"
}

# Request the data
# Note: In a real scenario we might wait to know exact bike dates, but here we j
print("Sending request to Weather API...")
weather_responses = openmeteo.weather_api(url, params=params)
raw_weather_response = weather_responses[0]
print("Weather data received.")
```

1.3 Knicks Data Acquisition

We use Selenium to scrape the game schedule from ESPN.

In [ ]:
```
import re
import time
import pandas as pd
from bs4 import BeautifulSoup, Comment

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC


START_URL = "https://www.basketball-reference.com/teams/NYK/2025_games.html"


# ----------------------------
# 0️⃣ Driver setup
# ----------------------------
```

```
def make_driver(headless=False):
    options = webdriver.ChromeOptions()
    if headless:
        options.add_argument("--headless=new")
    options.add_argument("--window-size=1400,900")
    options.add_argument("--no-sandbox")
    options.add_argument("--disable-dev-shm-usage")
    return webdriver.Chrome(options=options)


# ------------------------------
# 1️⃣ Cookie banner (IT/EN)
# ------------------------------
def accept_cookies_if_present(driver, timeout=8):
    xpaths = [
        "//button[contains(., 'Accetta tutto')]",
        "//button[contains(., 'Accept All')]",
        "//button[contains(., 'Accept all')]",
        "//button[contains(., 'Salva')]",
        "//button[contains(., 'Save')]",
        "//button[contains(., 'Rifiutare')]",
        "//button[contains(., 'Reject')]",
        "//a[contains(., 'Accetta tutto')]",
        "//a[contains(., 'Accept All')]",
    ]
    for xp in xpaths:
        try:
            btn = WebDriverWait(driver, timeout).until(
                EC.element_to_be_clickable((By.XPATH, xp))
            )
            driver.execute_script("arguments[0].scrollIntoView({block:'center'})
            btn.click()
            time.sleep(0.7)
            print("✅ Cookie banner chiuso")
            return True
        except Exception:
            pass
    print("ℹ️ Cookie banner non presente o già chiuso")
    return False


# ------------------------------
# 2️⃣ Uncomment HTML (BRef tables in <!-- -->)
# ------------------------------
def uncomment_html(html: str) -> str:
    soup = BeautifulSoup(html, "lxml")
    for c in soup.find_all(string=lambda t: isinstance(t, Comment)):
        if "<table" in c:
            try:
                frag = BeautifulSoup(c, "lxml")
                c.replace_with(frag)
            except Exception:
                pass
    return str(soup)


# ------------------------------
# 3️⃣ Discover season links dynamically from current page
# ------------------------------
def discover_season_links(page_html: str):
```

```
    html2 = uncomment_html(page_html)
    soup = BeautifulSoup(html2, "lxml")

    links = {}
    for a in soup.select('a[href*="/teams/NYK/"][href$="_games.html"]'):
        href = a.get("href")
        if not href:
            continue
        m = re.search(r"/teams/NYK/(\d{4})_games\.html", href)
        if m:
            year = int(m.group(1))
            links[year] = href  # dedup per anno

    return dict(sorted(links.items(), key=lambda x: x[0]))


# ----------------------------
# 4️⃣ Extract RAW games table exactly as loaded
# ----------------------------
def extract_games_df_raw(page_html: str) -> pd.DataFrame:
    html2 = uncomment_html(page_html)
    tables = pd.read_html(html2, attrs={"id": "games"})
    if not tables:
        raise ValueError("Tabella id='games' non trovata.")
    df = tables[0].copy()

    # unica pulizia "tecnica": rimuove intestazioni ripetute dentro la tabella
    if "Rk" in df.columns:
        df = df[df["Rk"].astype(str).str.match(r"^\d+$", na=False)].copy()

    # NON FACCIO ALTRO: no parsing date, no stripping '@', no location, no dtype
    return df


# ----------------------------
# 5️⃣ Main: dynamic click-based scraping from START_URL
# ----------------------------
def scrape_dynamic_raw_from_start(start_url: str, years_to_scrape=(2025, 2026),
    driver = make_driver(headless=headless)
    wait = WebDriverWait(driver, 20)

    frames = []  # lista di DF grezzi per stagione

    try:
        # open start page
        driver.get(start_url)
        wait.until(EC.presence_of_element_located((By.TAG_NAME, "body")))
        accept_cookies_if_present(driver)

        # discover season links from start page
        season_map = discover_season_links(driver.page_source)

        missing = [y for y in years_to_scrape if y not in season_map]
        if missing:
            raise RuntimeError(
                f"Non trovo i link per queste stagioni: {missing}. "
                f"Stagioni disponibili trovate: {list(season_map.keys())}"
            )

        # loop seasons (click-based)
```

```
        for year in years_to_scrape:
            href = season_map[year]
            print(f"\n➡️ Scraping RAW stagione {year} (click su {href})")

            # return to start page to ensure link is present to click
            driver.get(start_url)
            wait.until(EC.presence_of_element_located((By.TAG_NAME, "body")))
            accept_cookies_if_present(driver)

            link_el = wait.until(EC.element_to_be_clickable((By.XPATH, f'//a[@hr
            driver.execute_script("arguments[0].scrollIntoView({block:'center'})
            link_el.click()

            wait.until(EC.url_contains(href))
            time.sleep(1.0)

            df_raw = extract_games_df_raw(driver.page_source)

            # aggiungo SOLO metadata, senza toccare le colonne BRef
            df_raw.insert(0, "season_year", year)

            frames.append(df_raw)
            print(f"✅ Righe RAW salvate: {len(df_raw)}")

            time.sleep(1.0)

        return pd.concat(frames, ignore_index=True)

    finally:
        driver.quit()


# ----------------------------
# 6️⃣ RUN + save
# ----------------------------
if __name__ == "__main__":
    years = (2025, 2026)  # cambia qui se vuoi
    df_raw_all = scrape_dynamic_raw_from_start(
        start_url=START_URL,
        years_to_scrape=years,
        headless=False
    )

    df_raw_all.to_csv("knicks_bref_RAW_dynamic.csv", index=False)
    print(f"\n✅ Salvato: knicks_bref_RAW_dynamic.csv | righe totali: {len(df_r
```

1.4 Holidays Data Acquisition

We initialize the Holidays library for New York.

In [ ]:
```
print("--- Acquiring Holidays Data ---")
# We don't know the exact years yet until we process bike data,
# but we can acquire the library object.
ny_holidays_obj = holidays.US(subdiv='NY')
```

2. Pre-processing and Cleaning

Now we take the raw data and clean it up for analysis

2.1 Bike Data Cleaning

We iterate through files, filter bad trips, and aggregate by hour.

In [ ]:
```
import glob
import pandas as pd
import numpy as np

print("--- Step 1: Bike Data ---")

# Cerca tutti i CSV con 'citibike' nel nome
file_list = glob.glob("*-citibike-tripdata_*.csv")
all_hourly_data = []

# Loop su ogni file
for file in file_list:
    print("Reading file:", file)

    try:
        df = pd.read_csv(file, usecols=['started_at', 'ended_at', 'member_casual

        # Converti le colonne tempo in datetime
        df['started_at'] = pd.to_datetime(df['started_at'])
        df['ended_at'] = pd.to_datetime(df['ended_at'])

        # Calcola la durata del viaggio in minuti
        df['duration_min'] = (df['ended_at'] - df['started_at']).dt.total_second

        # Mantieni solo viaggi tra 1 minuto e 3 ore
        df = df[(df['duration_min'] > 1) & (df['duration_min'] < 180)].copy()

        # Conta membri e casual
        df['member_count'] = np.where(df['member_casual'].str.contains('member|S
        df['casual_count'] = np.where(df['member_casual'].str.contains('casual|C

        #creazione colonna data e ora
        df['date']=df['started_at'].dt.date
        df['time_hour']=df['started_at'].dt.floor('h')
        df['time_hour']=df['time_hour'].dt.strftime('%H:%M')

        # Aggregazione per ora (somme)
        hourly_stats = df.groupby(['date', 'time_hour']).agg(
            total_trips=('started_at', 'count'),
            sum_duration_min=('duration_min', 'sum'),
            member_count=('member_count', 'sum'),
            casual_count=('casual_count', 'sum')
        ).reset_index()

        # Aggiungi alla lista
        all_hourly_data.append(hourly_stats)

    except Exception as e:
        print("Could not read file:", file)
        print("Reason:", e)
```

```
# Combina tutti i dati in un unico DataFrame
print("Combining all data...")
df_main = (
    pd.concat(all_hourly_data)
      .groupby(['date', 'time_hour'], as_index=False)
      .sum()
)

# Calcola la durata media finale
df_main['avg_duration_min'] = df_main['sum_duration_min'] / df_main['total_trips

# Ordina per data/ora
df_main = df_main.sort_values(['date', 'time_hour'])

print("Bike data ready! Rows:", len(df_main))
print("Max average duration:", df_main['avg_duration_min'].max())
```

2.2 Weather Data Cleaning

We process the API response into a clean DataFrame and handle timezones.

In [ ]:
```
print("--- Cleaning Weather Data ---")
# Process the object (raw_weather_response) from Step 1.2
hourly = raw_weather_response.Hourly()
hourly_data = {"date": pd.date_range(
    start = pd.to_datetime(hourly.Time(), unit = "s", utc = True),
    end = pd.to_datetime(hourly.TimeEnd(), unit = "s", utc = True),
    freq = pd.Timedelta(seconds = hourly.Interval()),
    inclusive = "left"
)}

hourly_data["temp_c"] = np.round(hourly.Variables(0).ValuesAsNumpy(), 1)  #tempe
hourly_data["humidity"] = np.round(hourly.Variables(1).ValuesAsNumpy(), 1)
hourly_data["temp_apparent_c"] = np.round(hourly.Variables(2).ValuesAsNumpy(), 1
hourly_data["rain_mm"] = np.round(hourly.Variables(3).ValuesAsNumpy(), 1) #rain
hourly_data["weather_code"] = hourly.Variables(4).ValuesAsNumpy()   #type_weathe
hourly_data["wind_kmh"] = np.round(hourly.Variables(5).ValuesAsNumpy() * 3.6, 1)

df_weather = pd.DataFrame(data = hourly_data)
df_weather['date'] = df_weather['date'].dt.tz_localize(None) # Remove time zone
```

2.3 Knicks Data Cleaning

We ensure dates are correct and categorize games as Home or Away.

In [ ]:
```
import pandas as pd
import numpy as np

def add_date_time_home_away_from_bref(df: pd.DataFrame) -> pd.DataFrame:
    out = df.copy()

    # -------------------------
    # 1) HOME / AWAY
    # Nel tuo dataset l'away marker è in "Unnamed: 5" con valore "@"
    # -------------------------
    if "Unnamed: 5" not in out.columns:
```

```
        raise KeyError("Non trovo la colonna 'Unnamed: 5' (marker '@' away).")

    away_marker = out["Unnamed: 5"].astype(str).str.strip()
    out["home_away"] = np.where(away_marker.eq("@"), "away", "home")

    # -------------------------
    # 2) DATA -> AAAA/MM/GG
    # Date esempio: "Tue, Oct 22, 2024"
    # -------------------------
    if "Date" not in out.columns:
        raise KeyError("Non trovo la colonna 'Date'.")

    out["date_dt"] = pd.to_datetime(out["Date"], errors="coerce")
    out["date_yyyy_mm_dd"] = out["date_dt"].dt.strftime("%Y/%m/%d")

    # -------------------------
    # 3) ORARIO -> 24h (HH:MM)
    # Start (ET) esempio: "7:30p", "7:00p"
    # -------------------------
    if "Start (ET)" not in out.columns:
        raise KeyError("Non trovo la colonna 'Start (ET)'.")

    start_raw = out["Start (ET)"].astype(str).str.strip()

    # Converte "7:30p" -> "7:30 PM", "11:00a" -> "11:00 AM"
    start_norm = (start_raw
                  .str.replace(r"(?i)\s*tbd\s*", "", regex=True)
                  .str.replace(r"(?i)a$", " AM", regex=True)
                  .str.replace(r"(?i)p$", " PM", regex=True)
                  .str.strip())

    t = pd.to_datetime(start_norm, format="%I:%M %p", errors="coerce")
    out["time_24h"] = t.dt.strftime("%H:%M")

    return out



df2 = add_date_time_home_away_from_bref(df_raw_all)
```

In [ ]:

```
# tieni SOLO queste colonne (ordine garantito)
keep_cols = [
    "date_yyyy_mm_dd",
    "time_24h",
    "home_away",
    "Opponent",
    "Tm",
    "Opp",
    "W",
    "L",
    "Streak",
    "LOG"
]

df_final = df2.loc[:, keep_cols].copy()
```

2.4 Holidays and Events Cleaning

We prepare the specific event dictionary and holiday list.

In [ ]:
```
print("--- Cleaning Holidays Data ---")
# Update holidays object with actual years from bike data
if not df_main.empty:
    years = df_main['date_hour'].dt.year.unique()
    ny_holidays = holidays.US(subdiv='NY', years=years)
else:
    ny_holidays = holidays.US(subdiv='NY')

major_events_dict = {
    "2024-11-03": "NYC Marathon 2024",
    "2024-11-28": "Thanksgiving Parade",
    "2024-12-31": "New Year's Eve",
    "2025-05-04": "Five Boro Bike Tour",
    "2025-11-02": "NYC Marathon 2025"
}
```

3. Investigation of Correspondences

We check if the datasets cover the same time periods to ensure a good merge.

In [ ]:
```
print("--- Checking Data Overlaps ---")
print("Bike Data Range:", df_main['date_hour'].min(), "to", df_main['date_hour']
print("Weather Data Range:", df_weather['date'].min(), "to", df_weather['date'].
if not df_knicks.empty:
    print("Knicks Data Range:", df_knicks['game_date'].min(), "to", df_knicks['g
else:
    print("Knicks Data: Empty")

# Check if weather covers all bike hours
missing_weather = df_main[~df_main['date_hour'].isin(df_weather['date'])]
if not missing_weather.empty:
    print("Warning: Some bike hours are missing weather data!")
else:
    print("Good news: Weather data covers all bike trips.")
```

4. Mapping Rules and Conflict Resolution

We decide how to join the data:

- **Bike Data** is our primary table.
- **Weather** will be joined by EXACT match on Hour (Inner Join).
- **Knicks/Holidays** will be joined by Date (Left Join), filling '0' or 'None' where no event exists.

5. Integration and Storage

We execute the merges and save the final result.

In [ ]:
```
print("--- Integration ---")
# 5.1 Merge Weather
df_final = pd.merge(df_main, df_weather, left_on='date_hour', right_on='date', h
df_final.drop(columns=['date'], inplace=True)

# 5.2 Merge Knicks
df_final['temp_date_only'] = df_final['date_hour'].dt.floor('d')
df_final = pd.merge(df_final, df_knicks, left_on='temp_date_only', right_on='gam
df_final['is_knicks_home'] = df_final['is_knicks_home'].fillna(0).astype(int)
df_final['opponent'] = df_final['opponent'].fillna("No Game")
df_final.drop(columns=['temp_date_only', 'game_date'], inplace=True)

# 5.3 Map Holidays & Events
df_final['is_holiday'] = df_final['date_hour'].apply(lambda x: 1 if x in ny_holi
df_final['holiday_name'] = df_final['date_hour'].apply(lambda x: ny_holidays.get

df_final['major_event'] = df_final['date_hour'].apply(lambda x: major_events_dic
df_final['is_major_event'] = np.where(df_final['major_event'] != "None", 1, 0)

# 5.4 Save
output_filename = "NYC_Full_Analysis_Integrated.csv"
print(f"Saving final integrated dataset to {output_filename}...")
df_final.to_csv(output_filename, index=False)
print("Integration Complete!")
print(df_final.head())
```